

# Building a Commercial Cross-Platform VPN App

WireGuard · OpenVPN · Custom — a Flutter implementation guide

A practical architecture and build guide for a subscription-based VPN product with a Flutter front-end and native tunnel engines. Covers all three protocol approaches in depth, plus backend, servers, billing, and store/legal reality.

Flutter (iOS + Android)

Commercial product

WireGuard

OpenVPN

Custom / Obfuscated

---

**Target:** iOS + Android first (Flutter), desktop later • **Goal:** Commercial, subscription-based • **Date:** 2026

---

# 1. The one thing to understand before writing any code

A VPN app has four distinct pieces. People conflate them and get lost. Keep them separate:

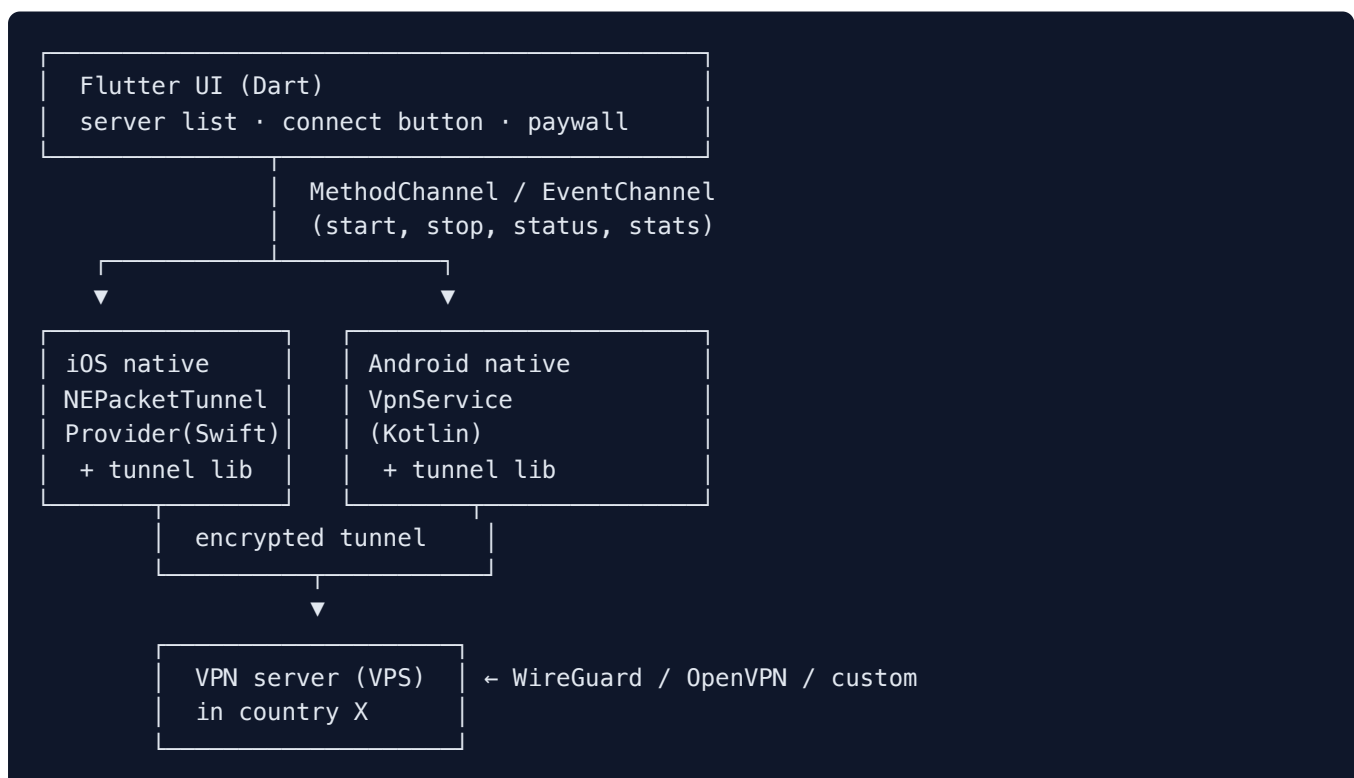
| Piece         | What it does                                     | Where it runs       | Build it?                                            |
|---------------|--------------------------------------------------|---------------------|------------------------------------------------------|
| Client UI     | Connect button, server list, account, paywall    | Flutter (Dart)      | ✅ Yes                                                |
| Tunnel engine | Actually encrypts & routes device traffic        | Native OS extension | ⚠️ Integrate an existing engine — don't write crypto |
| VPN servers   | Exit nodes traffic flows out of                  | VPS in datacenters  | ✅ Provision/configure                                |
| Backend API   | Accounts, auth, subscriptions, hands out configs | Your cloud          | ✅ Yes                                                |

## The Flutter constraint (critical)

**Dart code cannot capture or tunnel OS network traffic.** Operating systems only grant packet capture to a sandboxed *system network extension*:

- **iOS / macOS** → Apple `NetworkExtension` (`NEPacketTunnelProvider`), requires the **Network Extensions** entitlement (must be requested from Apple).
- **Android** → `android.net.VpnService` (a bound foreground service), requires `BIND_VPN_SERVICE` and a user consent dialog.

So the real architecture of "a Flutter VPN" is:



The **tunnel engine** is the part that differs between WireGuard, OpenVPN, and custom. **That is the choice this document is really about.** Everything else (UI, backend, servers, billing) is roughly the same regardless.

## 2. Approach A — WireGuard RECOMMENDED DEFAULT

### 2.1 What it is

A modern VPN protocol: ~4,000 lines of code, fixed modern crypto (Curve25519, ChaCha20-Poly1305, BLAKE2s), UDP-only, extremely fast and battery-friendly. The current industry default for new consumer VPNs.

### 2.2 How the pieces map

**Server side (each VPS):** install `wireguard` / `wg-quick`. Each server has a static keypair; each client is a "peer" with its own keypair and internal IP. A client config:

```
[Interface]
PrivateKey = <client_private_key>
Address    = 10.7.0.5/32
DNS        = 10.7.0.1

[Peer]
PublicKey  = <server_public_key>
Endpoint   = 185.x.x.x:51820
AllowedIPs = 0.0.0.0/0, ::/0      # route ALL traffic through tunnel
PersistentKeepalive = 25
```

**Client side (native):** don't write the protocol — use the official embeddable libraries: `wireguard-apple` ( `WireGuardKit` ) inside `NEPacketTunnelProvider`, and `wireguard-android` inside `VpnService`. For Flutter, `wireguard_flutter` / `wireguard_dart` wraps both. For a commercial app you'll likely fork it for custom status, stats, and per-app routing.

### 2.3 Connection flow

1. User taps Connect in Flutter.
2. Flutter asks **your backend** for a config for server X for this user.
3. Backend verifies the subscription, generates/reuses a client keypair, registers the public key as a peer on the server, returns the `.conf`.
4. Flutter passes the config down the MethodChannel to native.
5. Native extension boots the WireGuard tunnel; the OS shows the VPN icon.
6. Native streams status + byte counters up an EventChannel; Flutter renders them.

### 2.4 Pros / cons

-  Fastest, lowest battery, simplest config, best mobile UX, easy peer automation.
-  Best-supported embeddable libraries.
-  **UDP-only** → easy to block on restrictive networks. Mitigate with obfuscation (\$4) or OpenVPN-TCP fallback.

- ⚠️ You manage IP assignment & key rotation. Tools: **wg-easy**, **Netmaker**, **Firezone**.

**Recommendation: Ship WireGuard first.** Shortest path to a fast, working product.

## 3. Approach B — OpenVPN

### 3.1 What it is

The older, battle-tested protocol. Runs over **TCP or UDP**, uses OpenSSL/mbedTLS with X.509 certificates. Slower/heavier than WireGuard but far better at *evading blocks* (TCP/443 OpenVPN looks like normal HTTPS).

### 3.2 How the pieces map

**Server side:** run an OpenVPN daemon (or **OpenVPN Access Server** for a managed control plane). Each client gets a `.ovpn` profile:

```
client
dev tun
proto udp           # or tcp for stealth
remote 185.x.x.x 1194
cipher AES-256-GCM
auth SHA256
<ca>...</ca>
<cert>...</cert>
<key>...</key>
```

**Client side (native):** iOS/macOS use `OpenVPNAdapter` or `TunnelKit`; Android uses the `ics-openvpn` core (`de.blinkt.openvpn`). Flutter: `openvpn_flutter` wraps both.

### 3.3 Pros / cons

- ✅ TCP mode + port 443 = very good at getting through restrictive firewalls.
- ✅ Extremely mature, well audited, universally supported; rich auth (certs, user/pass, 2FA).
- ⚠️ Slower, higher battery/CPU than WireGuard.
- ⚠️ Config & PKI (certificate authority) management is more involved.

**Recommendation: Add OpenVPN as your "stealth / fallback" protocol** after WireGuard works, so the app auto-switches when WireGuard is blocked.

## 4. Approach C — Custom protocol

### 4.1 What "custom" actually means (three honest levels)

**Do not invent your own encryption.** Rolling your own crypto is how commercial VPNs get breached. "Custom" in a serious product means one of these:

**Level 1 — Obfuscation wrapper around an existing protocol (recommended form of "custom")**

Keep WireGuard/OpenVPN doing the crypto, but wrap packets so censors can't fingerprint them:

- **wstunnel** — tunnel WireGuard/OpenVPN over WebSocket/HTTPS.
- **Shadowsocks / V2Ray / XRay (VLESS/VMess/Trojan)** — designed to look like ordinary TLS web traffic; heavily used against national firewalls.
- **obfs4 / Cloak** — pluggable transports that disguise the tunnel.
- **AmneziaWG** — a WireGuard fork with built-in obfuscation (junk packets, custom headers) that defeats DPI while keeping WireGuard's speed. **The best "custom-feeling" option today.**

## Level 2 — Custom transport, standard crypto

Build your own daemon using a vetted crypto library ( `libsodium` , BoringSSL, Noise Protocol Framework), but with your own packet format, handshake, multiplexing, port-hopping, or QUIC transport. What companies do for a differentiated "stealth" mode they fully control.

## Level 3 — Fully bespoke protocol

Only justified at scale with a dedicated security team and external audits. Not where you start.

### 4.2 How you'd build a Level-2 custom protocol

**Server:** a Go daemon that (1) opens a UDP/QUIC or TCP/TLS listener, (2) performs a **Noise Protocol** handshake for mutual auth + forward secrecy, (3) reads/writes a TUN device to route packets with NAT.

**Client:** your native extension reads packets off the OS TUN interface and hands them to your client core. Ship the Go core to *both* platforms via `gomobile bind` — one shared library for iOS and Android.

```
Flutter UI → native extension → [your Go client core via gomobile]
                (reads OS TUN)      Noise handshake + your framing
                                   encrypt → QUIC/UDP → your server daemon
```

### 4.3 Pros / cons

-  Full control; a real differentiator ("works where others are blocked").
-  You own the anti-censorship story.
-  You own **all** the security risk — needs audits, a threat model, careful key management.
-  Much slower to build and maintain.

**Recommendation:** Don't ship Level-2/3 custom at v1. Ship **WireGuard + OpenVPN**, then add **AmneziaWG or a wstunnel/XRay layer** as your "custom stealth mode." 90% of the marketing benefit for 10% of the risk.

## 5. Side-by-side decision table

| Dimension                  | WireGuard | OpenVPN          | Custom (obfuscated)   |
|----------------------------|-----------|------------------|-----------------------|
| Speed                      | ★★★★★     | ★★★              | ★★★★–★★★★             |
| Battery                    | ★★★★★     | ★★               | ★★★                   |
| Firewall evasion           | ★★        | ★★★★ (TCP/443)   | ★★★★★                 |
| Build effort               | Low       | Medium           | High                  |
| Security risk you own      | Low       | Low              | High                  |
| Library maturity (Flutter) | Good      | Good             | You build it          |
| Best role                  | Default   | Stealth fallback | Premium "unblockable" |

**Product strategy:** WireGuard by default → auto-fallback to OpenVPN-TCP/443 when UDP is blocked  
→ offer obfuscated mode (AmneziaWG / XRay) as a premium feature.

## 6. The rest of the product (same for all three protocols)

### 6.1 Backend API

Responsibilities: signup/login, subscription state, server list, and **issuing tunnel configs on demand**.  
Suggested stack: **Go or Node.js + PostgreSQL + Redis**.

```
POST /auth/register    POST /auth/login    # accounts (or Sign in with Apple)
GET  /servers          # id, country, city, load, protocols
POST /connect          # {serverId, protocol} → returns a fresh tunnel config
POST /disconnect       # deregister peer / revoke short-lived cert
GET  /subscription/status # entitlement check
POST /webhooks/revenuecat # billing events → flip entitlements
```

Config issuance is the security-sensitive core: verify entitlement, generate short-lived credentials, register them on the target server, return them. Rotate/revoke on disconnect and on subscription lapse.

### 6.2 Server fleet

- Rent VPS in the countries you advertise; pick hosts with VPN-friendly acceptable-use terms and good capacity.
- Automate with **Terraform + Ansible** so "a new city" is one command.
- Run a small **control agent** per server for add/remove peer (or wg-easy/Netmaker/Firezone for WG, OpenVPN AS for OpenVPN).
- **DNS:** run your own resolver per server (block leaks; optional ad/tracker blocking as a selling point).

### 6.3 Billing & subscriptions (RevenueCat)

- Use **RevenueCat** ( `purchases_flutter` ) to manage App Store + Play Store subscriptions with one SDK and a single "is this user entitled?" answer.

- Flow: user buys → RevenueCat validates the receipt → app reads the **entitlement** → RevenueCat **webhook** hits your backend → backend flips the subscription flag → `/connect` honors requests.
- Never gate access purely client-side; the backend must verify entitlement before issuing configs.

## 6.4 Flutter app structure

```
lib/
  main.dart
  core/          # DI, config, secure storage (flutter_secure_storage)
  data/          # backend API client, RevenueCat wrapper
  features/
    onboarding/  server_list/  connection/  paywall/  settings/
  platform/
    vpn_channel.dart # MethodChannel + EventChannel to native
  android/ ...VpnService + tunnel lib (Kotlin)
  ios/      ...NEPacketTunnelProvider extension + tunnel lib (Swift)
```

State: Riverpod or Bloc. The connection state machine ( `disconnected` → `connecting` → `connected` → `disconnecting` → `error` ) lives in Dart, driven by native events.

## 6.5 Must-have features users expect

Kill switch · Auto-connect on untrusted Wi-Fi · Split tunneling (Android native; iOS limited) · Protocol switch + auto-fallback · DNS/IPv6-leak protection · a genuine **no-logs** stance.

## 7. Store, legal & operational reality (don't skip)

- **Apple:** VPN apps need the **Network Extensions** entitlement and must be published by an **organization** Developer account (not individual). Review is strict on privacy.
- **Google Play:** must use `VpnService`, declare it, complete extra VPN/privacy declarations, no deceptive interception.
- **Privacy policy + no-logs:** legally required and scrutinized — say what you log and mean it.
- **Jurisdiction** affects logging obligations and marketing claims — get legal advice before launch.
- **Abuse handling:** exit nodes attract DMCA/fraud complaints — you need an abuse-response process or hosts drop you.

## 8. Recommended build order (phased)

**Phase 0 — Prototype (1 server, WireGuard, no billing).** Spin up one WireGuard VPS by hand; hard-code one client config; integrate `wireguard_flutter`; get Connect/Disconnect + live byte counters working on a real Android device, then iOS. *This proves the hardest part end to end.*

**Phase 1 — Backend + dynamic configs.** Build auth + `/servers` + `/connect` (dynamic WireGuard peer creation); app fetches from backend instead of hard-coding.

**Phase 2 — Monetization.** Integrate RevenueCat, build the paywall, gate `/connect` on entitlement via webhook.

**Phase 3 — Scale & resilience.** Terraform/Ansible the fleet to multiple countries; add OpenVPN fallback + auto-fallback logic.

**Phase 4 — Differentiate.** Add obfuscated/"custom" mode (AmneziaWG or XRay) as a premium feature; kill switch, auto-connect, split tunneling, polish, store submission.

## 9. Concrete starting libraries (Flutter)

| Need                            | Package / lib                                                                                            |
|---------------------------------|----------------------------------------------------------------------------------------------------------|
| WireGuard tunnel                | <code>wireguard_flutter</code> / <code>wireguard_dart</code> (wraps wireguard-apple + wireguard-android) |
| OpenVPN tunnel                  | <code>openvpn_flutter</code> (wraps TunnelKit/OpenVPNAdapter + ics-openvpn)                              |
| Custom Go core → both platforms | <code>gomobile_bind</code>                                                                               |
| Billing                         | <code>purchases_flutter</code> (RevenueCat)                                                              |
| Secure key/token storage        | <code>flutter_secure_storage</code>                                                                      |
| State                           | <code>flutter_riverpod</code> or <code>flutter_bloc</code>                                               |
| Backend                         | Go ( <code>wireguard-go</code> , <code>wgctrl</code> ) or Node.js + Postgres                             |
| Fleet automation                | Terraform + Ansible; wg-easy / Netmaker / Firezone; OpenVPN AS                                           |

## 10. TL;DR

- A "Flutter VPN" = **Flutter UI + native OS tunnel extension + your servers + your backend**. Dart can't tunnel; the native extension does.
- **Ship WireGuard first** — fastest path to a real product.
- **Add OpenVPN** as the TCP/443 stealth fallback.
- **"Custom" = an obfuscation layer (AmneziaWG/XRay/wstunnel) over vetted crypto**, not homemade encryption. Add it as a premium differentiator, not at v1.
- The non-tunnel plumbing (auth, server fleet, RevenueCat billing, no-logs policy, store entitlements) is the same for all three and is where most of the work lives.